



**S.B. JAIN INSTITUTE OF TECHNOLOGY
MANAGEMENT & RESEARCH, NAGPUR**

**Practical 11
Postlab (OPEN ENDED PRACTICAL)**

Aim: To implement and compare different CPU scheduling algorithms (FCFS and SJF) and analyze their performance based on waiting time and turnaround time.

Name: Swayam Bobade

USN:CM24053

Semester / Year:4th/2nd

Academic Session:2025-26

Date of Performance:

Date of Submission:

❖ **Aim:** To implement and compare different CPU scheduling algorithms (FCFS and SJF) and analyze their performance based on waiting time and turnaround time.

❖ **Objectives:**

- To understand CPU scheduling algorithms.
- To calculate Waiting Time and Turnaround Time.
- To compare performance of FCFS and SJF.

❖ **Requirements:**

✓ **Hardware Requirements:**

- Processor: Minimum 1 GHz
- RAM: 512 MB or higher
- Storage: 100 MB free space

✓ **Software Requirements:**

- Operating System: Linux/Unix-based
- Shell: Bash 4.0 or higher
- Text Editor: Nano, Vim, or any preferred editor

❖ **Theory:**

CPU Scheduling is a fundamental function of an operating system in which it decides **which process will be executed next by the CPU**. Since multiple processes are present in the system at the same time, proper scheduling ensures that all processes get a fair chance to execute. It plays a crucial role in achieving **efficient CPU utilization, reduced waiting time, and improved overall system performance**.

In a multiprogramming environment, several processes reside in the **ready queue**, waiting for CPU allocation. The operating system uses various scheduling algorithms to select one process among them. The choice of scheduling algorithm directly affects system efficiency, responsiveness, and throughput.

CPU scheduling becomes especially important in systems where multiple users or applications are running simultaneously, such as in computers, mobile devices, and servers. A good scheduling algorithm minimizes idle CPU time and ensures smooth execution of processes.

Types of Scheduling Algorithms Used:

1. FCFS (First Come First Serve)

FCFS is the simplest CPU scheduling algorithm. In this method, the process that arrives first in the ready queue is executed first. It follows the principle of a queue (FIFO – First In First Out).

- Easy to understand and implement
 - No starvation (every process gets a chance)
 - However, it may lead to **high waiting time**
 - Suffers from **convoy effect**, where short processes wait for long processes
-

2. SJF (Shortest Job First)

SJF selects the process with the **smallest burst time** for execution. It is more efficient than FCFS in terms of minimizing waiting time.

- Provides **minimum average waiting time**
- More efficient and faster compared to FCFS
- Difficult to predict exact burst time in real scenarios
- May cause **starvation** of longer processes

Important Terms

1. Waiting Time (WT)

Waiting Time is the total time a process spends waiting in the ready queue before getting CPU access.

Formula:

$$WT = \text{Start Time} - \text{Arrival Time}$$

It is an important parameter used to evaluate the efficiency of scheduling algorithms. Lower waiting time indicates better performance.

2. Turnaround Time (TAT)

Turnaround Time is the total time taken by a process from its arrival to its completion.

Formula:

$$TAT = \text{Completion Time} - \text{Arrival Time}$$

It includes both execution time and waiting time. Lower turnaround time means faster process completion and better system efficiency.

❖ **CODE:**

```
#include <stdio.h>

// FCFS Function
void fcfs(int n, int at[], int bt[]) {
    int ct[n], wt[n], tat[n];

    ct[0] = at[0] + bt[0];

    for(int i = 1; i < n; i++) {
        if(ct[i-1] < at[i])
            ct[i] = at[i] + bt[i];
        else
            ct[i] = ct[i-1] + bt[i];
    }

    for(int i = 0; i < n; i++) {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
    }

    float avg_wt = 0, avg_tat = 0;

    printf("\n--- FCFS Scheduling ---\n");
    printf("P\tAT\tBT\tCT\tTAT\tWT\n");

    for(int i = 0; i < n; i++) {
        avg_wt += wt[i];
        avg_tat += tat[i];
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
            i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
    }

    printf("Average WT = %.2f\n", avg_wt/n);
    printf("Average TAT = %.2f\n", avg_tat/n);
}

// SJF Function (Non-Preemptive)
void sjf(int n, int at[], int bt[]) {
    int ct[n], wt[n], tat[n], completed[n];

    for(int i = 0; i < n; i++)
        completed[i] = 0;
```

```
int time = 0, done = 0;

while(done < n) {
    int min = 9999, index = -1;

    for(int i = 0; i < n; i++) {
        if(at[i] <= time && completed[i] == 0) {
            if(bt[i] < min) {
                min = bt[i];
                index = i;
            }
        }
    }

    if(index == -1) {
        time++;
    } else {
        time += bt[index];
        ct[index] = time;
        tat[index] = ct[index] - at[index];
        wt[index] = tat[index] - bt[index];
        completed[index] = 1;
        done++;
    }
}

float avg_wt = 0, avg_tat = 0;

printf("\n--- SJF Scheduling ---\n");
printf("P\tAT\tBT\tCT\tTAT\tWT\n");

for(int i = 0; i < n; i++) {
    avg_wt += wt[i];
    avg_tat += tat[i];
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
}

printf("Average WT = %.2f\n", avg_wt/n);
printf("Average TAT = %.2f\n", avg_tat/n);
}
```

// Main

```
int main() {  
    int n = 4;  
  
    int at[] = {0, 1, 2, 3};  
    int bt[] = {6, 8, 7, 3};  
  
    fcfs(n, at, bt);  
    sjf(n, at, bt);  
  
    return 0;  
}
```

❖ **Output:**

Output

--- FCFS Scheduling ---

P	AT	BT	CT	TAT	WT
P1	0	6	6	6	0
P2	1	8	14	13	5
P3	2	7	21	19	12
P4	3	3	24	21	18

Average WT = 8.75

Average TAT = 14.75

--- SJF Scheduling ---

P	AT	BT	CT	TAT	WT
P1	0	6	6	6	0
P2	1	8	24	23	15
P3	2	7	16	14	7
P4	3	3	9	6	3

Average WT = 6.25

Average TAT = 12.25

=== Code Execution Successful ===

❖ **Conclusion:** CPU scheduling is essential for optimal system performance. Different algorithms have different advantages and limitations. While FCFS is simple, SJF provides better efficiency. The choice of scheduling algorithm depends on system requirements and workload conditions.

❖ **Discussion Questions:**

1. What is CPU Scheduling?
2. What is the main difference between FCFS and SJF?
3. What is Waiting Time?
4. Why is SJF considered better than FCFS?
5. What is starvation in CPU scheduling?

❖ **References:**

<https://www.geeksforgeeks.org/disk-scheduling-algorithms/>
<https://chatgpt.com/>

Date: ____ / ____ /2026

Signature
Course Coordinator
B.Tech CSE(AIML)
Sem: 4 / 2025-26